

REPORTE DE RESILIENCIA, BENCHMARK Y EVALUACION

PRUEBAS DE SISTEMAS EN VIVO Y VALIDACIÓN DE MÉTRICAS (GEMINI 3.5)

1. Detalle del Caso de Prueba: Falla de Base de Datos (Opcion A)

Durante la demostracion en vivo, se simulo una falla fisica de hardware bloqueando los permisos de escritura del archivo de base de datos local 'mock_db.json' (modo 'Solo lectura' en Windows).

- Comportamiento SRE Observado: El adaptador de FastAPI intercepto inmediatamente el error de disco (PermissionError) y, aplicando el patron de Transaccionalidad Estricta, aborto el flujo de ejecucion antes de realizar la llamada a Gemini 3.5 en Google, previniendo discrepancias y costos de tokens inutilis.
- Respuesta del Servidor: Se retorno un codigo HTTP 503 Service Unavailable limpio e informativo al usuario final, indicandole que la transaccion no pudo registrarse y que debe volver a intentar.
- Registro de Observabilidad: OpenTelemetry marco el Span 'simulate_transaction' con estado 'ERROR' e inyecto los metadatos de la excepcion de forma automatica en los logs del servidor.

2. Guia de Pruebas de Calidad y Latencias (Opcion B)

Para comprobar la velocidad, consistencia y calidad de tu API local, se definen dos comandos esenciales paso a paso para ejecutar ante el docente:

A) Correr la Suite de Pruebas Unitarias Offline (Pytest):

Comando: `python -m pytest codigo/tests/test_trading_api.py -v`

- Ejecuta 6 pruebas deterministas locales de autenticacion, enmascaramiento, salud y fallas en 1s.

B) Correr el Script de Benchmark de Rendimiento (Métricas p50 / p95):

Comando: `python codigo/scripts/benchmark_latencia.py`

- Realiza 10 llamadas secuenciales y computa de forma matematica los percentiles de tu API. Los resultados obtenidos registran un p50 (Mediana) de ~20.97 ms y un p95 (Peor caso del 5%) de ~28.86 ms.

3. Guia de Observabilidad de LLM Ops (Opcion C)

Para mostrar un monitoreo sofisticado de auditoria de prompts, tokens y costos en dolares en la nube:

A) Crear el Dataset en Langfuse Cloud:

Comando: `python codigo/scripts/create_langfuse_dataset.py`

- Crea de forma automatica el dataset 'trading_agent_evaluation' con 3 casos de prueba en la nube.

B) Correr la Corrida de Evaluacion en Vivo:

Comando: `python codigo/scripts/run_langfuse_evaluation.py`

- Ejecuta los casos del dataset con tu Gemini 3.5 real, enmascara el nombre del usuario y registra las trazas y generaciones en Langfuse, permitiendo calificar los resultados en la pestaña 'Runs'.

Documento de resiliencia y evaluacion de API - Marcelo Gilardoni | Solutions Architect.